Does Dimensionality Reduction Help Classical Intrusion Detection? Original Features versus PCA and Kernel PCA on a Class-Stratified CIC-IDS2017 Sample

[AUTHOR NAME 1 – TO BE FILLED][1] and [AUTHOR NAME 2 – TO BE FILLED][1]

[DEPARTMENT, INSTITUTION, CITY, COUNTRY – TO BE FILLED]
[EMAIL -- TO BE FILLED]

Abstract. Kernel principal component analysis is often paired with classical classifiers for network intrusion detection, but gains are usually reported as single-split accuracy without a no-reduction baseline. This study compares the 78 original flow features with principal component analysis and kernel principal component analysis (linear, radial basis function and cosine kernels; 5 to 30 components) across five classifiers. The data are a 6,000-record sample drawn from all eight CIC-IDS2017 files after exact-duplicate removal. The sample is class-stratified, with at least 100 records per class, and does not preserve natural traffic prevalence. Imputation, scaling and reduction are fitted on training folds only. Evaluation combines repeated stratified cross-validation, nested cross-validation, a small nested hyper-parameter search, two row-order robustness checks serving as a proxy for chronology (the files contain no timestamps), and a runtime and memory benchmark. A random forest on the original features achieved the highest observed macro-averaged F1 score, 0.901 ± 0.020 (accuracy 0.972 ± 0.004; nested estimate 0.900 ± 0.022). Accuracy hides weak detection of two web attack classes (F1 0.53 and 0.59). Original features outperformed both reductions for the decision tree and random forest and matched them at the best component count for logistic regression and k-nearest neighbors; only the support vector machine benefited from reduction, and its advantage fell from 0.076 to 0.021 after tuning. Kernel analysis took more than 200 times longer to fit than principal component analysis and about 37 times more traced memory. The findings are limited to one sample of one dataset.

Keywords: Network intrusion detection · CIC-IDS2017 · Kernel PCA · Dimensionality reduction · Class imbalance · Machine learning. *Code and results:* [REPOSITORY URL / ARCHIVE DOI – TO BE FILLED BY THE AUTHORS]

1 Introduction

Researchers often develop and compare network intrusion detection systems (NIDS) on public flow datasets [20], and CIC-IDS2017 [22] is one of the most

used. In the MachineLearningCVE files its flow records have 78 numeric features. That number invites dimensionality reduction, either to cut training cost or in the hope of better generalization. Linear PCA and auto-encoders have been tried on CIC-IDS2017 [3]. Kernel PCA (KPCA) [21] runs PCA in an implicit nonlinear feature space and has been paired with SVMs for intrusion detection [14].

Three problems make these comparisons hard to read. CIC-IDS2017 is imbalanced and, in its original form, has labelling and flow-construction errors [13, 16], so the benign majority can dominate accuracy. Security-oriented machine learning is also prone to pitfalls that inflate results [5]. And a claim that reduction helps needs a no-reduction baseline; without one, the benefit is assumed, not measured.

Research questions. RQ1: how does reduction change multi-class detection performance? **RQ2:** does KPCA beat the original space or ordinary PCA? **RQ3:** how do kernel, number of components and classifier interact? **RQ4:** what does reduction cost in time and memory? **RQ5:** do the conclusions hold under repeated class-aware validation, nested selection, tuning, row-order splits (a proxy for chronology) and smaller training sets?

Contributions. (i) A reproducible comparison of the original features, PCA and three KPCA kernels over five classical classifiers, on a sample taken from the raw files after exact same-label duplicate removal. It uses fold-wise fitting, a majority-class baseline, a nested-CV check of selection optimism and a small nested hyper-parameter search. (ii) A class-aware analysis of how far accuracy overstates detection quality. (iii) Paired statistical tests with an explicit independence caveat, plus two row-order robustness checks (proxy chronology). (iv) A runtime and memory comparison of KPCA with PCA. We propose no new algorithm. Code and results are described in the Data and Code Availability section.

2 Related Work

We surveyed fifteen studies. Table 1 lists five recent (2025) ones with algorithm, dataset and result *as stated by their authors*; the other ten, on the dataset and its problems, dimensionality reduction and evaluation practice, are discussed below. Their numbers come from different datasets, splits and metrics, so they are not a ranking.

Dataset and its problems. Ring et al. review public network-intrusion datasets and define 15 properties for judging which evaluation settings each dataset fits [20]. CIC-IDS2017 was introduced by Sharafaldin et al. [22]. Engelen et al. report problems in it: "TCP appendix" artefact flows (about a quarter of all flows), attack attempts labelled as attacks although they contain no malicious payload, and feature-extraction flaws that enable shortcut learning; they released a corrected version [13]. Panigrahi and Borah likewise describe shortcomings and propose a combined dataset [16]. We use the *original* labelled flows, so absolute scores may partly reflect such artefacts.

Dimensionality reduction and feature selection. Abdulhammed et al. apply an auto-encoder and PCA to CIC-IDS2017 (81 to 10 dimensions) and report about 99.6 % accuracy [3]; Kuang et al. combine KPCA, an SVM and a genetic algorithm [14]. Abdel-Rahman et al. state that a feature-importance-guided autoencoder outperforms earlier dimensionality-reduction techniques on three datasets including CIC-IDS2017 [1]. Chavan and Alone report "high accuracy" for a random forest on CIC-IDS 2017 without a figure or split in the abstract [7], and Patil et al. include PCA among their reduction methods [17]. Most report accuracy on one split; none that we read pairs a no-reduction baseline, per-class analysis and fold-wise fitting across PCA and several KPCA kernels.

Evaluation practice. Arp et al. describe pitfalls in machine learning for security that can lead to unrealistic performance [5], and Apruzzese et al. propose cross-evaluating ML-based NIDS across datasets [4]. From them we took three practices: every data-dependent step is fitted on training folds only, accuracy is always reported next to class-aware metrics, and every reduction is compared with a trivial baseline and a no-reduction baseline. This is not a systematic survey. IEEE Xplore was unreachable, so all studies above come from open publisher or preprint pages.

3 Dataset and Preprocessing

Data. We use the eight daily CIC-IDS2017 `MachineLearningCVE` files: 2,830,743 flows, 78 numeric features, 15 labels (counts computed from the files; no provider checksum was available). There is no timestamp column. Rows with missing or infinite values are 0.2 % or less of each file (inf $\rightarrow$ NaN) and were not imputed before sampling. Eight features are constant in every file and were kept to match the 78-feature definition of earlier work.

Duplicates. 308,381 flows (10.9 %) are exact duplicates (identical on all 78 features and the label) of another flow and were removed before sampling, because identical flows in training and test folds inflate scores. Feature-identical flows with *different* labels were not removed: 698 such conflicting-label groups remain in the 2.52 M pool. None of them fell into the 6,000-record sample: it contains no two records with identical feature vectors, with the same or with different labels. Duplication is uneven: removal took 43 % of PortScan flows (158,930 to 90,819) and 25 % of DoS Hulk flows, versus 7.8 % of benign flows.

Sample. Exact KPCA needs an $n \times n$ kernel matrix, so we draw a stratified random sample of 6,000 records from the de-duplicated pool of 2,522,362 flows (seed 0): each class contributes at least 100 records (all of them if fewer exist) and the rest is allocated proportionally to pool prevalence. Heartbleed (11), SQL injection (21) and Infiltration (36) are therefore complete but tiny; the floor over-represents attack classes relative to real traffic (benign: 67.2 % of the sample versus 83 % of the pool). Sampling uses only labels and random numbers; *no imputation or scaling statistic is computed before the split.* Only two feature cells in the sample are missing; they are imputed with training-fold means inside

Table 1. Recent studies (2025) and this work. Results are quoted from each source and are not directly comparable (different datasets, sampling, splits and metrics).

Author (year)	Title	Algorithm	Dataset	Reported result
Waghmode et al. [23] (2025)	Intrusion detection system based on machine learning using least square support vector machine	Quantum-inspired LS-SVM with exhaustive feature selection	NSL-KDD, CIC-IDS-2017, UNSW-NB15	Accuracy 99.5 % on CIC-IDS-2017
Patil et al. [17] (2025)	Optimizing network intrusion detection systems through ensemble learning and feature selection using the CIC-IDS2017 dataset	Bagging and boosting ensembles; information gain, chi-square, PCA (20 features)	CIC-IDS2017, stratified 70/30 split	Accuracy 99.98 % (Extra Trees, 24 features); 99.58–99.96 % with PCA
Abdelaziz et al. [2] (2025)	Enhancing network threat detection with random forest-based NIDS and permutation feature importance	Random forest, permutation importance, undersampling; repeated stratified 5-fold CV	CIC-IDS2017	Weighted-F1 99.8 %, macro-F1 93.31 % (26 features)
Xu and Liu [24] (2025)	Robust anomaly detection in network traffic: evaluating machine learning models on CICIDS2017	MLP, 1D-CNN, one-class SVM, LOF	CICIDS2017	Accuracy 97.75 % (MLP, overall test set)
Emirmahmutoğlu and Atay [12] (2025)	A feature selection-driven machine learning framework for anomaly-based intrusion detection systems	PSO/FPA/DE feature selection; LR, DT, RF, k-NN and others	KDD CUP 99, NSL-KDD, UNSW-NB15, CSE-CIC-IDS2018	Accuracy 99.9965 % on CSE-CIC-IDS2018 (FPA, RF)
This work (2026)	(this paper)	Original features vs PCA vs linear/RBF/cosine KPCA; LR, k-NN, SVM, DT, RF	CIC-IDS2017, 6,000-record class-stratified sample	RF (original): accuracy 97.2 %, macro-F1 90.1 %

each fold. The sample also stores each record's source file (day) and original row position, used for the row-order checks below.

4 Methodology

Pipeline. Fig. 1 shows the protocol. Within each training fold we fit a mean imputer and a standard scaler, then the dimensionality reducer; test folds are only transformed. Feature spaces are: *original* (78 standardised features); *PCA*; and *KPCA* with linear, RBF (scikit-learn default $\gamma = 1/78$) and cosine kernels [21], each with $n \in \{5, 10, 15, 20, 30\}$ components. The reducer is fitted once per fold and shared by all classifiers, which is equivalent to fitting it inside each pipeline because it is deterministic (unit-tested).

Algorithm and formulas. Algorithm 1 summarises one evaluation of a feature space and classifier. Features are standardised as $z_j = (x_j - \mu_j)/\sigma_j$ with μ_j, σ_j from the training fold. PCA projects onto the leading eigenvectors $\mathbf{v}_k$

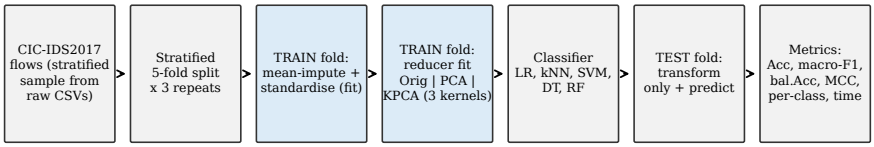


Fig. 1. Experimental pipeline. All data-dependent preprocessing and reduction is fitted on the training fold only.

of the training covariance matrix $\mathbf{C} = \frac{1}{n}\mathbf{Z}^\top\mathbf{Z}$. KPCA uses the kernel matrix $K_{ij} = k(\mathbf{z}_i, \mathbf{z}_j)$ with

$$k_{\text{lin}} = \mathbf{z}^\top\mathbf{z}', \qquad k_{\text{rbf}} = \exp(-\gamma\|\mathbf{z}-\mathbf{z}'\|^2),\ \gamma = \tfrac{1}{78}, \qquad k_{\cos} = \frac{\mathbf{z}^\top\mathbf{z}'}{\|\mathbf{z}\|\,\|\mathbf{z}'\|}, \quad (1)$$

centred as $\tilde{K} = K - \mathbf{1}_n K - K\mathbf{1}_n + \mathbf{1}_n K \mathbf{1}_n$; with $\mathbf{1}_n$ the $n \times n$ matrix whose entries are all $1/n$. For the leading eigenpairs $(\lambda_k, \boldsymbol{\alpha}_k)$ of $\tilde{K}$, with $\|\boldsymbol{\alpha}_k\| = 1$, a sample is projected to $\tilde{K}_{\text{new}}\boldsymbol{\alpha}_k/\sqrt{\lambda_k}$, with $\tilde{K}_{\text{new}}$ the centred kernel against the training fold (matches `KernelPCA.transform`; dividing by λ_k does not). With per-class precision P_c and recall R_c, the primary metric is

$$F1_c = \frac{2P_cR_c}{P_c + R_c}, \qquad \text{macro-F1} = \frac{1}{C}\sum_{c=1}^{C} F1_c, \quad C = 15, \quad (2)$$

and balanced accuracy equals macro-recall. The multi-class MCC from the confusion matrix $\mathbf{M}$ with s samples, t_k true and p_k predicted counts of class k, and $c = \sum_k M_{kk}$ correct predictions is

$$\text{MCC} = \frac{c\,s - \sum_k p_k t_k}{\sqrt{s^2 - \sum_k p_k^2}\,\sqrt{s^2 - \sum_k t_k^2}}. \quad (3)$$

Classifiers. Logistic regression (L2, lbfgs), k-NN with $k = 5$ [10], an SVM with RBF kernel, $C = 1$ and γ="scale" [9], a decision tree with entropy criterion [19], and a random forest with 100 trees [6], using scikit-learn [18]. The main grid uses these fixed default settings; a separate nested search (below) tests whether tuning changes the conclusions. Each classifier except k-NN is also run with `class_weight=balanced`. A majority-class classifier is the trivial baseline. Resampling (e.g. SMOTE) is not used: Heartbleed has only 8–9 training records per fold, too few for meaningful synthetic interpolation.

Validation. Repeated stratified 5-fold cross-validation, 3 repeats (15 folds), seed 42; every method sees identical folds, giving paired observations. Test folds hold about 2 Heartbleed and 4 SQL-injection records, so per-class estimates for these classes are noisy; we report means and standard deviations over the 15 folds.

Algorithm 1 Fold-wise evaluation of one feature space S and classifier f

Require: sample (X, y) of 6,000 records; space $S \in \{\text{original, PCA, KPCA}(k, n)\}$; classifier f; 3 repeats of stratified 5-fold CV (seed 42)
Ensure: mean and standard deviation of accuracy, macro-F1, MCC and cost over 15 folds
for each repeat r and fold (tr, te) **do**
  fit mean imputer and standard scaler on X_{tr}; transform X_{tr} and X_{te}
  if $S \neq$ original **then**
    fit reducer T_S (n components) on the scaled X_{tr} only
    $Z_{\text{tr}} \leftarrow T_S(X_{\text{tr}})$; $Z_{\text{te}} \leftarrow T_S(X_{\text{te}})$
  else
    $Z_{\text{tr}} \leftarrow X_{\text{tr}}$; $Z_{\text{te}} \leftarrow X_{\text{te}}$
  end if
  fit f on $(Z_{\text{tr}}, y_{\text{tr}})$; predict $\hat{y}$ on Z_{te}; record metrics and fit time
end for
return mean ± std over the 15 folds (nested selection repeats this inside each outer training set with 3 inner folds)

Nested cross-validation for selection. Picking the best of 189 configurations (21 feature spaces × 9 classifier and weighting variants) on the same folds that score it flatters the winner. To measure this, selection is nested: inside each of the 15 outer training sets (4,800 records) a stratified 3-fold inner cross-validation scores every configuration by macro-F1, the winner is chosen there, and only that winner is scored on the outer test fold it never saw. We run this for (A) the global choice and, for each classifier, for (C) the best *reduced* space with weighting versus (D) the original space with weighting chosen the same way.

Nested hyper-parameter search. For k-NN, SVM and random forest on four spaces (original, PCA-30, KPCA-cosine-30, KPCA-RBF-30), a 3-fold inner cross-validation inside each of the first five outer folds selects from small grids: $k \in \{3, 5, 7, 11\}$; SVM $C \in \{0.1, 1, 10\}$, $\gamma \in \{\text{scale}, 0.01, 0.1\}$, class weight none/balanced; random forest 100/200 trees, depth none/10/20, class weight none/balanced. The RBF width of the KPCA reducer itself is not tuned.

Row-order robustness checks (proxy chronology). The raw files have no timestamps; attack flows occur in contiguous row blocks, so we *assume*, without proof, that row order follows capture time. (i) *Head/tail*: the first 70 % of each file's rows train, the last 30 % test. (ii) *Blocked CV*: each file is cut into five contiguous blocks, each held out in turn. Only classes present in both training and test are scored, and a random split of the same rows on the same classes is the comparator.

Metrics. Accuracy; macro-precision, macro-recall and macro-F1; weighted-F1; balanced accuracy; and the multi-class Matthews correlation coefficient (MCC) as implemented in scikit-learn (Chicco and Jurman argue for MCC in the binary case [8]). **Macro-F1 is the primary model-selection criterion**, chosen before the runs because it weights all 15 classes equally. Per-class results and the confusion matrix use pooled out-of-fold predictions of the first repeat.

Statistics. Per classifier we compare feature spaces at matched $n = 10$ with two-sided Wilcoxon signed-rank tests on the 15 paired macro-F1 values, Holm-corrected over 30 comparisons, with the rank-biserial correlation r as effect size, and Friedman omnibus tests, following the non-parametric procedures of Demšar [11] (written for several data sets; here the blocks are folds of one sample). Because folds share training data, the 15 differences are positively correlated and naive p-values are too liberal [15]; we do not apply the Nadeau–Bengio correction and use the tests for screening, not as proof. Comparisons against each classifier's *best* reduced space are selected on the same folds, which favours the reduced space; the nested analysis removes that bias.

Cost. A serial benchmark (no parallel workers) records training and inference time (median of 5 repetitions, one fold split) and peak memory from Python's `tracemalloc`, measured in a separate pass. Native allocations inside compiled code are not traced, so memory figures are lower bounds most reliable for comparing reducers.

Setup. A 2-vCPU, 7 GB VM (Python 3.13, scikit-learn 1.9) ran the main grid (2,850 fits, about 28 min), nested CV (about 64 min) and tuning (about 16 min).

5 Results

5.1 Original-feature baseline and the cost of reading accuracy alone

Predicting the majority class gives accuracy 0.672 but macro-F1 0.054 and MCC 0.000. The random forest on original features reaches accuracy 0.972 ± 0.004 but macro-F1 0.901 ± 0.020 (weighted-F1 0.971, balanced accuracy 0.883, MCC 0.948 ± 0.008): accuracy exceeds macro-F1 by 0.071, and the gap comes from a few classes. Per class (Table 2), ten of the 15 classes, including benign, DDoS, the four DoS families, both Patator brute-force classes, PortScan and Heartbleed, have F1 ≥ 0.96. The weak points are web brute force (F1 0.53) and web XSS (0.59), which are largely mistaken for each other (48 of 103 brute-force records are predicted as XSS and 35 of 101 XSS records as brute force), then bot (0.87; recall 0.82), infiltration (0.875; recall 0.78, 36 records) and SQL injection (0.84; 21 records). Engelen et al. find that most web brute-force and XSS flows carry no payload in the forward direction [13], which may explain why they are hard to separate. These are the weak points that accuracy hides (Table 2).

5.2 Original space versus PCA versus Kernel PCA

Table 3 and Fig. 2 compare spaces at $n = 10$. Of the 30 comparisons, 23 had Holm-adjusted $p < 0.05$ (screening only). The original space was significantly better than PCA-10 for logistic regression, SVM, decision tree and random forest, and than RBF-10 and cosine-10 for all five classifiers; for k-NN it was indistinguishable from PCA-10 (Δ macro-F1 -0.001). Among the reduced spaces, RBF-KPCA was significantly worse than PCA for LR, k-NN, DT and RF, but

Table 2. Per-class results of the selected model (RF, original features), pooled out-of-fold predictions, first repeat.

Class	Precision	Recall	F1	Support
BENIGN	0.984	0.996	0.990	4034
Bot	0.924	0.817	0.867	104
DDoS	0.997	0.997	0.997	340
DoS GoldenEye	0.975	0.966	0.970	119
DoS Hulk	0.998	0.979	0.988	424
DoS Slowhttptest	0.955	0.973	0.964	110
DoS slowloris	1.000	0.927	0.962	110
FTP-Patator	1.000	1.000	1.000	111
Heartbleed	1.000	1.000	1.000	11
Infiltration	1.000	0.778	0.875	36
PortScan	0.989	0.981	0.985	270
SSH-Patator	1.000	0.943	0.971	106
Web Brute Force	0.581	0.485	0.529	103
Web Sql Injection	0.941	0.762	0.842	21
Web XSS	0.558	0.624	0.589	101

better than PCA for the SVM; cosine KPCA was significantly better than PCA for the SVM (+0.120), worse for LR, k-NN and DT, and not different for RF. Linear KPCA and PCA did not differ significantly for any classifier. The Friedman omnibus tests (spaces at $n = 10$ within each classifier; classifiers within each space) were all significant (all Holm-adjusted $p \leq 2.7 \times 10^{-9}$), but they inherit the fold-dependence caveat above and serve only as screening. Against each classifier's *best* reduced space (table in the repository), original features were clearly better for DT (+0.070) and RF (+0.056; $p_{\text{Holm}} = 0.0003$, better in all 15 folds); for logistic regression the best reduced space (PCA, 30 components) came within 0.004 ($p_{\text{Holm}} = 0.021$, a significant but negligible gap); k-NN showed no difference; and the SVM was the single clear case where reduction helped: cosine KPCA with 30 components raised macro-F1 from 0.675 to 0.746 ($p_{\text{Holm}} = 0.0003$, better in all 15 folds). That family of comparisons is selected on the same folds and therefore optimistic. The SVM gain needs many components: cosine KPCA reaches 0.605 at 10 components (below the original 0.675) and 0.746 at 30.

Table 3. Accuracy / macro-F1 (± std over 15 folds) at 10 components (original = 78 features), standard class weights.

Space	LR	KNN	SVM	DT	RF
Original (78)	0.924 / 0.736±0.022	0.934 / 0.800±0.024	0.906 / 0.675±0.017	0.964 / 0.881±0.021	0.972 / 0.901±0.020
PCA-10	0.842 / 0.506±0.018	0.932 / 0.802±0.023	0.842 / 0.485±0.024	0.941 / 0.805±0.025	0.956 / 0.834±0.022
KPCA-lin-10	0.842 / 0.507±0.017	0.932 / 0.802±0.023	0.842 / 0.485±0.024	0.942 / 0.806±0.024	0.957 / 0.836±0.019
KPCA-RBF-10	0.805 / 0.306±0.015	0.918 / 0.757±0.029	0.880 / 0.525±0.016	0.920 / 0.755±0.037	0.937 / 0.790±0.037
KPCA-cos-10	0.852 / 0.462±0.012	0.926 / 0.783±0.028	0.904 / 0.605±0.018	0.928 / 0.786±0.027	0.944 / 0.821±0.029

5.3 All executions in one chart

Fig. 2 places every classifier and feature space at 10 components in a single chart (25 executions, macro-F1, mean ± std over 15 folds). Within each classifier group the black bar is the original space; only for the SVM does a reduced space (cosine KPCA) exceed it, and the tree-based classifiers lead overall.

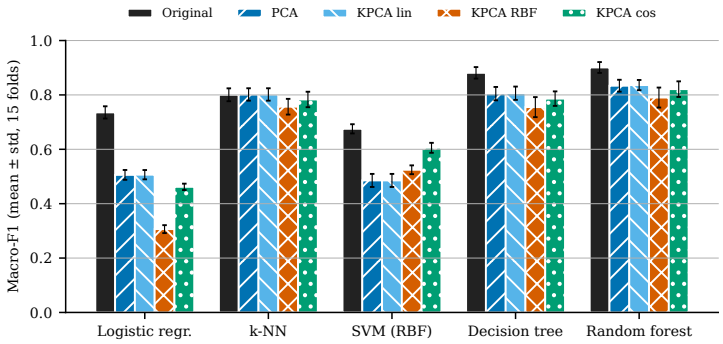


Fig. 2. Macro-F1 of all classifier × feature-space executions at 10 components (mean ± std over 15 folds; standard class weights; axis starts at zero).

5.4 Selection optimism: nested cross-validation

Selecting the best of 189 configurations on the scoring folds is optimistic, so selection was repeated inside a nested cross-validation (Section 4). The configuration chosen on inner folds reached macro-F1 0.900 ± 0.022 (accuracy 0.972, MCC 0.947) on the outer folds against 0.901 for the same-CV maximum: an optimism of about 0.001. The inner folds chose an original-feature configuration in all 15 outer folds (RF 11 times, DT 4). Original features were clearly better than the inner-selected best reduced space for DT (+0.093) and RF (+0.062), practically equal for logistic regression (+0.004, unadjusted $p = 0.010$, far below the fold standard deviation of 0.022) and k-NN (+0.002), and worse for the SVM (−0.060; cosine KPCA-30 chosen in all 15 folds). The p-values are unadjusted Wilcoxon tests over 15 outer folds that share training data.

5.5 Effect of hyper-parameter tuning

Table 4 reports the nested search (five outer folds). Tuning left the random forest unchanged (original 0.901 default, 0.899 tuned; best reduced 0.842 versus 0.839). For k-NN, tuning gave +0.007 on original features; the best reduced space was cosine KPCA-30 under defaults (gap 0.000) and PCA-30 after tuning (0.810 versus 0.808, gap −0.002; default gap in PCA-30: +0.003), always far

below fold variation. The SVM changed most: tuned macro-F1 rose from 0.668 to 0.791 on original features and from 0.745 to 0.812 with cosine KPCA-30, so the reduced-space advantage shrank from 0.076 to 0.021. Five folds cannot establish a 0.021 difference, and the selected values sit on grid edges (SVM $C = 10$ and k-NN $k = 3$ in all 20 selections), so we treat the SVM advantage of KPCA as a property of the default RBF-SVM that largely disappears with tuning, not as a robust benefit of KPCA.

Table 4. Nested hyper-parameter search: mean macro-F1 over five outer folds. For each setting (default, tuned) the best reduced space among PCA-30, KPCA-cos-30 and KPCA-RBF-30 is chosen by that setting's own score; Δ = original − reduced (negative: reduction better). K-cos-30 = KPCA, cosine kernel, 30 components. The last column gives the default-setting gap measured in the tuned-best reduced space.

	Default settings			Tuned (nested search)			
Clf	Orig.	Best reduced	Δ	Orig.	Best reduced	Δ	Δ def., tuned-best space
KNN	0.801	0.801 (K-cos-30)	+0.000	0.808	0.810 (PCA-30)	-0.002	+0.003
SVM	0.668	0.745 (K-cos-30)	-0.076	0.791	0.812 (K-cos-30)	-0.021	-0.076
RF	0.901	0.842 (PCA-30)	+0.059	0.899	0.839 (PCA-30)	+0.060	+0.059

5.6 Row-order robustness (proxy chronology)

In the head/tail split 4,199 records train and 1,769 test (32 test records of unseen classes were dropped); only seven of the 15 classes occur in both parts, and infiltration has four test records. The random forest on original features scored 0.838 versus 0.948 for a random split of the same rows and classes (decision tree 0.846 versus 0.959), a penalty of about 0.11. For DT, RF and LR the reduced spaces lost at least as much (RF: PCA-10 0.759, cosine KPCA-30 0.704), so the order original > reduced for DT, RF and LR was preserved; k-NN stayed a near tie (0.790 versus 0.806) and the SVM again favoured cosine KPCA-30 (0.616 versus 0.479). In the blocked cross-validation (6–10 classes scored per fold) the random forest scored 0.851 versus 0.932 for random folds on the same classes, with the original space again ahead for DT, RF and LR. These checks assume that row order follows capture time and score only a subset of classes, so they support the ranking but give no precise penalty (tables in the repository).

5.7 Kernel, component count and class weighting

Linear KPCA equals PCA on centred data: results were identical for k-NN and SVM and within 0.001 (LR) or 0.007 (trees, tiny eigen-solver differences, not tested), so a "linear KPCA" result is a PCA result. Component count mattered more than kernel for LR (macro-F1 0.37 at 5 PCA components to 0.73 at 30, versus 0.736 on original features) and SVM; for k-NN, DT and RF the kernel mattered slightly more, with small effects. RBF-KPCA with the default

γ was weakest for LR, k-NN, DT and RF; for the SVM cosine KPCA was best (mean macro-F1 0.657 versus 0.536 linear, 0.515 RBF; γ untuned). Balanced class weights raised balanced accuracy for LR and SVM but macro-F1 only for SVM and DT, and *lowered* it for LR (0.736 → 0.710) and RF (0.901 → 0.885); the best model stayed the unweighted RF.

5.8 Computational cost

The serial benchmark (table in the repository) at 4,800 training / 1,200 test records. KPCA fitting took 5.5–5.6 s against 0.02 s for PCA, increased decision-tree training from 0.07 s (PCA) to 5.3–5.6 s and random-forest training from 0.84–0.90 s to 6.3–6.6 s, and raised random-forest inference about fivefold (0.020–0.021 s to 0.092–0.123 s per 1,200 records). Peak traced memory was 363 MB for KPCA against 9.8 MB for PCA or no reduction, and grew roughly quadratically with n (3.1, 11.0, 41.6, 161.3, 359.1 MB for $n = 400$ to 4800; log–log slope 1.9), consistent with $n \times n$ float64 kernel matrices (184 MB at $n = 4800$). As a purely theoretical estimate (not a measurement), one dense float64 kernel matrix for all 2,830,743 source flows would occupy about 64 TB, so exact KPCA on the full dataset would need approximation or sub-sampling (not tested). Timings come from a shared 2-vCPU machine; differences below roughly a factor of two should not be over-interpreted, and no real-time claim is made.

5.9 Accuracy comparison with recent studies

Fig. 3 and Table 1 show reported accuracy side by side. The recent studies quote 97.75–99.9965 % on their own datasets and splits, whereas our random forest reaches 97.2 % accuracy but 0.901 macro-F1 on a sample that over-represents attacks. These numbers measure different things; they show only that accuracy near 100 % on imbalanced flow data can coexist with weak minority-class detection, as our per-class results show.

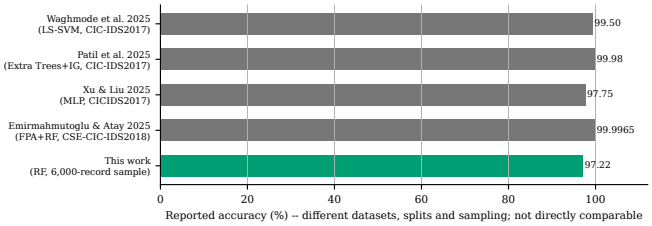


Fig. 3. Accuracy reported by recent studies versus this work. Datasets, sampling, splits and metrics differ; the figure is not a performance ranking.

6 Discussion

Answers to the research questions. RQ1 and RQ2: reducing the 78 standardised features to 5–30 PCA or KPCA components mostly did not help and usually lowered macro-F1. The loss was clear for the decision tree and random forest and negligible for logistic regression (30 components) and k-NN. The one gain, the RBF-SVM with cosine KPCA, largely disappeared under tuning (0.076 to 0.021). RQ3: for k-NN, DT and RF the kernel mattered slightly more than the number of components; for LR and SVM it was the other way round. Default-γ RBF was weakest for four of five classifiers, and linear KPCA equals PCA. RQ4: at $n = 4{,}800$, KPCA multiplied reducer fit time by more than 200 and traced memory by about 37 (memory grows quadratically), with no benefit for four of five classifiers. RQ5: orderings were stable across repeated folds, training sizes, nested selection, tuning and the scorable row-order splits.

Why accuracy misleads. Benign flows are 67.2 % of the sample, so a trivial classifier already scores 0.672 accuracy. The best model's accuracy of 0.972 goes with a macro-F1 of 0.901 and web-attack F1 values of only 0.53 and 0.59. Accuracy hides that failure on two classes. It also cannot tell us whether a one-point difference is noise; the fold standard deviation of macro-F1 is about 0.02.

Untested explanation. A few leading components may discard discriminative low-variance directions.

7 Threats to Validity and Limitations

The conclusions depend on the limitations below.

1. Sample, not full data. All results come from *one* stratified sample of 6,000 records out of 2.5 M de-duplicated flows (0.24 %), because exact KPCA is quadratic in memory. The sample has a floor of 100 records per class and is *not* prevalence-preserving: attack classes are over-represented, so accuracy, macro-F1 and class priors do not describe deployment-time prevalence. Heartbleed (11 records), SQL injection (21) and infiltration (36) are complete but tiny, so their per-class estimates are very uncertain, and another random sample could change rankings that differ by a few points. We did not repeat the pipeline over several sampling seeds.

2. Leakage status. Sampling was performed on the raw, un-imputed data, and imputation, scaling and reduction are fitted inside each training fold. Remaining possible leakage channels were *not* eliminated: records from the same attack session or burst can still fall into both training and test folds in the random 5-fold design, and near-duplicate (not exact-duplicate) flows are not removed. The row-order checks reduce but do not remove this concern, and they rest on the unproven assumption that row order follows capture time.

3. Limited external validity. All data come from one dataset captured in one testbed over one week. The row-order checks score only seven classes (head/tail split) or six to ten classes per fold (blocked) and use a single split or five folds; there is no cross-dataset evaluation. These conclusions hold only for

random and within-file row-ordered splits of CIC-IDS2017, not for drift over real time.

4. Selection and inference. The headline configuration (RF, original features) was the best of 189 on the same cross-validation that produced its score. The nested analysis shows the optimism is about 0.001 macro-F1 here, but the nested estimate itself is conditioned on the same sample. Repeated-CV significance tests violate independence [15]; p-values are screening evidence, and unadjusted values are reported for the nested comparisons.

5. Limited tuning. Hyper-parameter search covered only k-NN, SVM and random forest, four spaces and five outer folds, with small grids whose optimum lay on the edge for k-NN and SVM C. Logistic regression, the decision tree and the KPCA kernel parameters (γ for RBF) were not tuned, so conclusions about those are about default settings. Class weighting was the only imbalance treatment; resampling was not evaluated.

6. Dataset quality and cost. The original CIC-IDS2017 flows contain known labelling and feature-construction artefacts [13], which may distort scores (and may explain the web brute-force/XSS confusion); we did not use the corrected release. Timings and memory come from one 2-vCPU virtual machine, memory is a traced lower bound, and the 64 TB figure is arithmetic, not a measurement.

We propose no new algorithm and claim no state-of-the-art, real-time, production-ready or generalizable performance. The supported finding is narrower: on this sample, original features were at least as good as PCA or KPCA for four of five classifiers, the one exception largely vanished with tuning, and accuracy hid a systematic failure on web attacks.

8 Conclusion

On a 6,000-record sample drawn from all eight CIC-IDS2017 files after exact same-label duplicate removal, with imputation, scaling and reduction fitted inside each training fold, a random forest on the original 78 features scored highest: macro-F1 0.901 ± 0.020, accuracy 0.972 ± 0.004, and 0.900 ± 0.022 with nested selection. PCA and KPCA with 5–30 components mostly did not beat the original features. They were clearly worse for the decision tree and random forest and practically equal for logistic regression (30 components) and k-NN. Only the RBF-SVM gained, with cosine KPCA, and that advantage shrank from 0.076 to 0.021 after tuning. The ranking also held in the two row-order checks, which rest on the unproven assumption that row order follows capture time. Linear KPCA coincided with PCA. At 4,800 records KPCA needed more than 200 times the fit time and about 37 times the traced memory. Accuracy (0.972) concealed weak detection of web brute force and XSS (F1 0.53, 0.59). The results apply to this sample and to CIC-IDS2017 only.

Future work. Repeat the pipeline over several sampling seeds and larger samples with an approximate KPCA; add cross-dataset evaluation; widen the

tuning grids and tune the KPCA kernel parameters; test the corrected CIC-IDS2017 release [13]; and evaluate session-grouped splits.

Data and Code Availability

The raw CIC-IDS2017 MachineLearningCVE files are distributed by the dataset provider [22] and are not redistributed here. The sampling script, the sampling manifest (file names, raw and de-duplicated class counts, seed), all experiment, analysis and claim-checking code, per-fold raw results and the generated tables and figures are available at [REPOSITORY URL / ARCHIVE DOI - TO BE FILLED BY THE AUTHORS]; the file `RUN_EXPERIMENTS.md` lists the commands.

Acknowledgments and AI-assistance disclosure

[THE AUTHORS MUST CONFIRM THAT THIS DISCLOSURE IS ACCURATE AND COMPLETE, AND ADD ANY OTHER TOOL USED, BEFORE SUBMISSION.] Generative AI was used in this work: Claude (Anthropic; Claude Code, model Claude Sonnet 5.5) helped (i) write and revise the Python pipeline (data preparation, cross-validation, nested CV, tuning, row-order checks, statistics, runtime benchmark, tables, figures and an automated claim checker); (ii) draft and edit the text of all sections, including the abstract, captions and limitations; and (iii) look up and cross-check bibliographic metadata and statements attributed to cited sources. All reported numbers were produced by code run on the data described and re-checked by script. The authors reviewed the content and remain fully responsible for it. [FUNDING / ACKNOWLEDGMENTS – TO BE FILLED BY THE AUTHORS.] The authors declare no competing interests [TO BE CONFIRMED BY THE AUTHORS].

References

1. Abdel-Rahman, M.A., et al.: Feature importance guided autoencoder for dimensionality reduction in intrusion detection systems. Scientific Reports **16**, 5013 (2026). https://doi.org/10.1038/s41598-026-36695-9
2. Abdelaziz, M.T., Radwan, A., Mamdouh, H., et al.: Enhancing network threat detection with random forest-based NIDS and permutation feature importance. Journal of Network and Systems Management **33**, 2 (2025). https://doi.org/10.1007/s10922-024-09874-0
3. Abdulhammed, R., Musafer, H., Alessa, A., Faezipour, M., Abuzneid, A.: Features dimensionality reduction approaches for machine learning based network intrusion detection. Electronics **8**(3), 322 (2019)
4. Apruzzese, G., Pajola, L., Conti, M.: The cross-evaluation of machine learning-based network intrusion detection systems. IEEE Transactions on Network and Service Management **19**(4), 5152–5169 (2022). https://doi.org/10.1109/TNSM.2022.3157344

5. Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K.: Dos and don'ts of machine learning in computer security. In: Proc. 31st USENIX Security Symposium. pp. 3971–3988 (2022)
6. Breiman, L.: Random forests. Machine Learning **45**(1), 5–32 (2001)
7. Chavan, P.V., Alone, N.V.: Optimizing intrusion detection with random forest: A high-accuracy approach using CIC-IDS 2017. International Journal of Computer Applications **187**(3) (2025). https://doi.org/10.5120/ijca2025924816
8. Chicco, D., Jurman, G.: The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. BMC Genomics **21**, 6 (2020)
9. Cortes, C., Vapnik, V.: Support-vector networks. Machine Learning **20**(3), 273–297 (1995)
10. Cover, T., Hart, P.: Nearest neighbor pattern classification. IEEE Transactions on Information Theory **13**(1), 21–27 (1967)
11. Demšar, J.: Statistical comparisons of classifiers over multiple data sets. Journal of Machine Learning Research **7**, 1–30 (2006)
12. Emirmahmutoğlu, E., Atay, Y.: A feature selection-driven machine learning framework for anomaly-based intrusion detection systems. Peer-to-Peer Networking and Applications **18**, 161 (2025). https://doi.org/10.1007/s12083-025-01947-4
13. Engelen, G., Rimmer, V., Joosen, W.: Troubleshooting an intrusion detection dataset: the CICIDS2017 case study. In: Proc. IEEE Security and Privacy Workshops (SPW). pp. 7–12 (2021). https://doi.org/10.1109/SPW53761.2021.00009
14. Kuang, F., Xu, W., Zhang, S.: A novel hybrid KPCA and SVM with GA model for intrusion detection. Applied Soft Computing **18**, 178–184 (2014)
15. Nadeau, C., Bengio, Y.: Inference for the generalization error. Machine Learning **52**(3), 239–281 (2003)
16. Panigrahi, R., Borah, S.: A detailed analysis of CICIDS2017 dataset for designing intrusion detection systems. International Journal of Engineering & Technology **7**(3.24), 479–482 (2018)
17. Patil, D.R., Pattewar, T.M., Shinde, T.S., Kumavat, K.S., Deshpande, S.N.: Optimizing network intrusion detection systems through ensemble learning and feature selection using the CIC-IDS2017 dataset. Informatica **49**, 641–670 (2025). https://doi.org/10.31449/inf.v49i4.7678
18. Pedregosa, F., et al.: Scikit-learn: Machine learning in Python. Journal of Machine Learning Research **12**, 2825–2830 (2011)
19. Quinlan, J.R.: Induction of decision trees. Machine Learning **1**(1), 81–106 (1986)
20. Ring, M., Wunderlich, S., Scheuring, D., Landes, D., Hotho, A.: A survey of network-based intrusion detection data sets. Computers & Security **86**, 147–167 (2019). https://doi.org/10.1016/j.cose.2019.06.005
21. Schölkopf, B., Smola, A., Müller, K.R.: Nonlinear component analysis as a kernel eigenvalue problem. Neural Computation **10**(5), 1299–1319 (1998)
22. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP). pp. 108–116 (2018)
23. Waghmode, P., et al.: Intrusion detection system based on machine learning using least square support vector machine. Scientific Reports **15**, 12066 (2025). https://doi.org/10.1038/s41598-025-95621-7
24. Xu, Z., Liu, Y.: Robust anomaly detection in network traffic: Evaluating machine learning models on CICIDS2017. arXiv:2506.19877 [cs.CR], v2 (2025)